North South University

Department of Electrical & Computer Engineering

PROJECT REPORT

Password-Based Digital Lock System

Course Code: CSE231

Course Title: Digital Logic Design Lab

Faculty: Dr. Nusrat Jahan Lisa

Date of Submission: 30.04.2026

Section: 3

Group Number:		
SL no.	Group Members Name	ID
1.	Syed Tahsin Reza	2511769042
2.	Gazi Mainul Kabir Jabid	2413709642
3.	Hasan Mohammad Abdul Qadir	2312411042

Objective

The primary goal of this project is to design and implement a password-based digital security system using sequential logic concepts. The system simulates a real-world digital lock that recognizes a specific 4-bit binary code, tracks failed attempts, and triggers a lockout functionality after three consecutive incorrect entries.

Components Used

To build this digital lock, the following hardware components are utilized:

- **ICs:** IC-74HC74 D Flip-Flops (for state memory and LED latching)

  IC-7408 (AND Gate 2 Input)

  IC-7432 (OR Gate 2 Input)

  IC-7411 (AND Gate 3 Input)

  IC-7404 (Hex-Inverter)

- **LEDs:** Green (Correct), Red (Incorrect), and Yellow (System Locked).
- **Switches:** DIP switches for the 4-bit binary input (S3, S2, S1, S0) and 2 push-button for ENTER (Clock) and Reset.
- **Breadboard:** For hardware circuit implementation.
- **Wires:** Jumper wires for inter-component connections.

System Description

The system is a sequential logic mechanism that verifies a 4-bit binary input against a predefined password: **1001**.

- **Input:** The user sets a 4-bit code using switches and submits it by pressing the **ENTER** button, which acts as the clock signal.
- **Verification:** The system compares the input bits. If they match **1001**, the **Green LED** activates. If there is any mismatch, the **Red LED** turns on and the system logs a "wrong attempt".

- **Lock Condition:** The system keeps track of failures. Upon the 3rd consecutive incorrect attempt, the system enters a **LOCK state**. In this state, the **Yellow LED** stays lit, and the system ignores all further inputs until the **RESET** signal is triggered.

Logic Equations Used:

- **Match Logic:** Match = S3.S2'S1'.S0 (Detects 1001)
- **Wrong Logic:** Wrong = Match'
- **State Transitions:**
    - D0 = Wrong.Q0’ + Q1.Q0
    - D1 = Wrong.(Q1 + Q0) + Q1.Q0

Another flip flop for LED Memory:

D = match

Clock = ENTER button

Q = stored_match

Led output

Green = not_locked · stored_match

Red = not_locked · stored_wrong

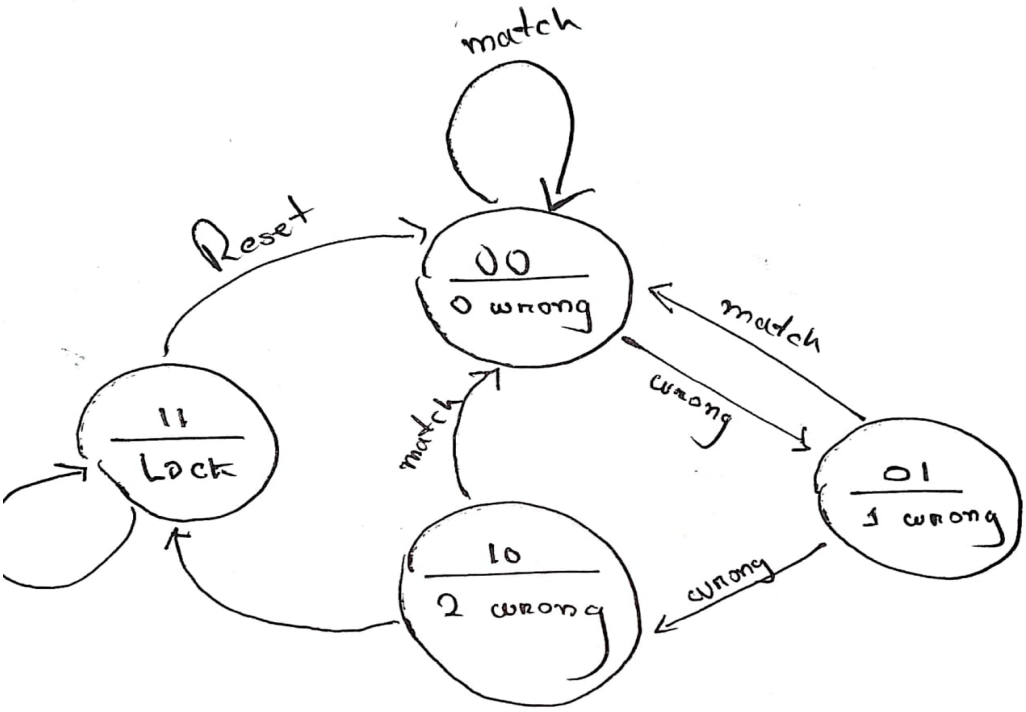
Yellow = Q1 · Q0

Design Section (a) State Table

Q1	Q0	Current Meaning	Next State (if Wrong)	Next State (if Match)
0	0	0 wrong attempts	01	00

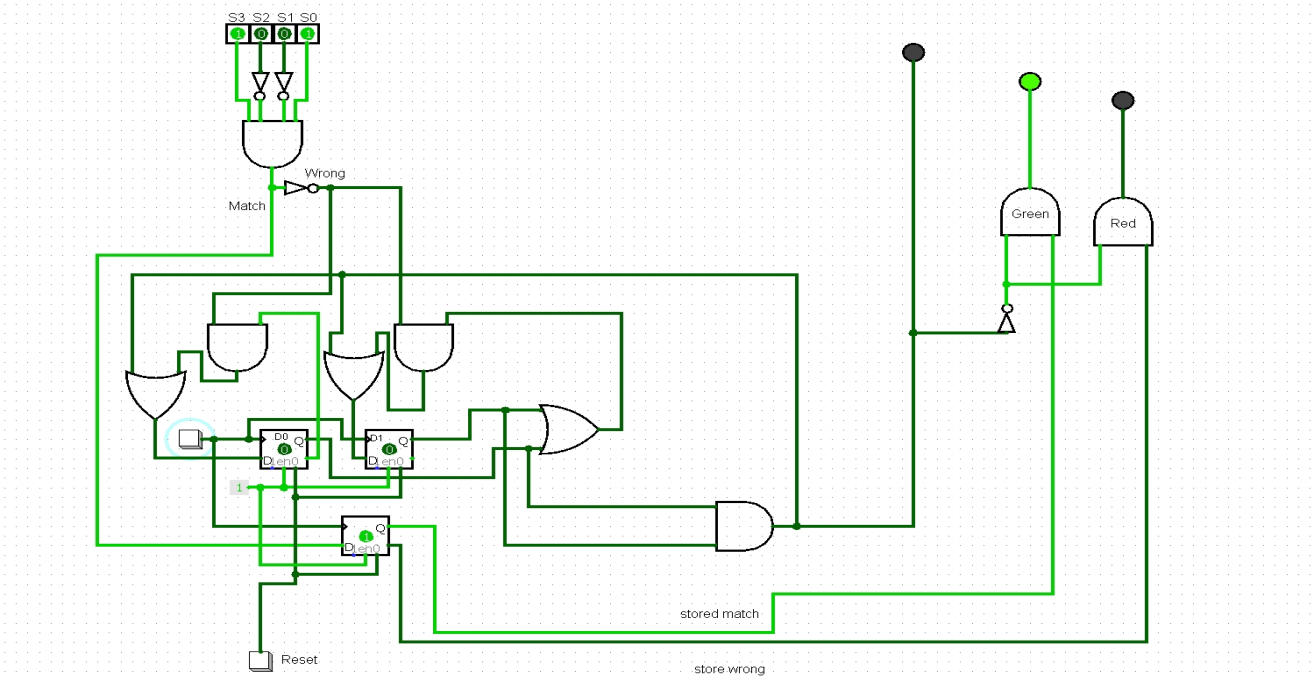
0	1	1 wrong attempt	10	00
1	0	2 wrong attempts	11 (LOCK)	00
1	1	**LOCK**	11	11

(b) State Diagram

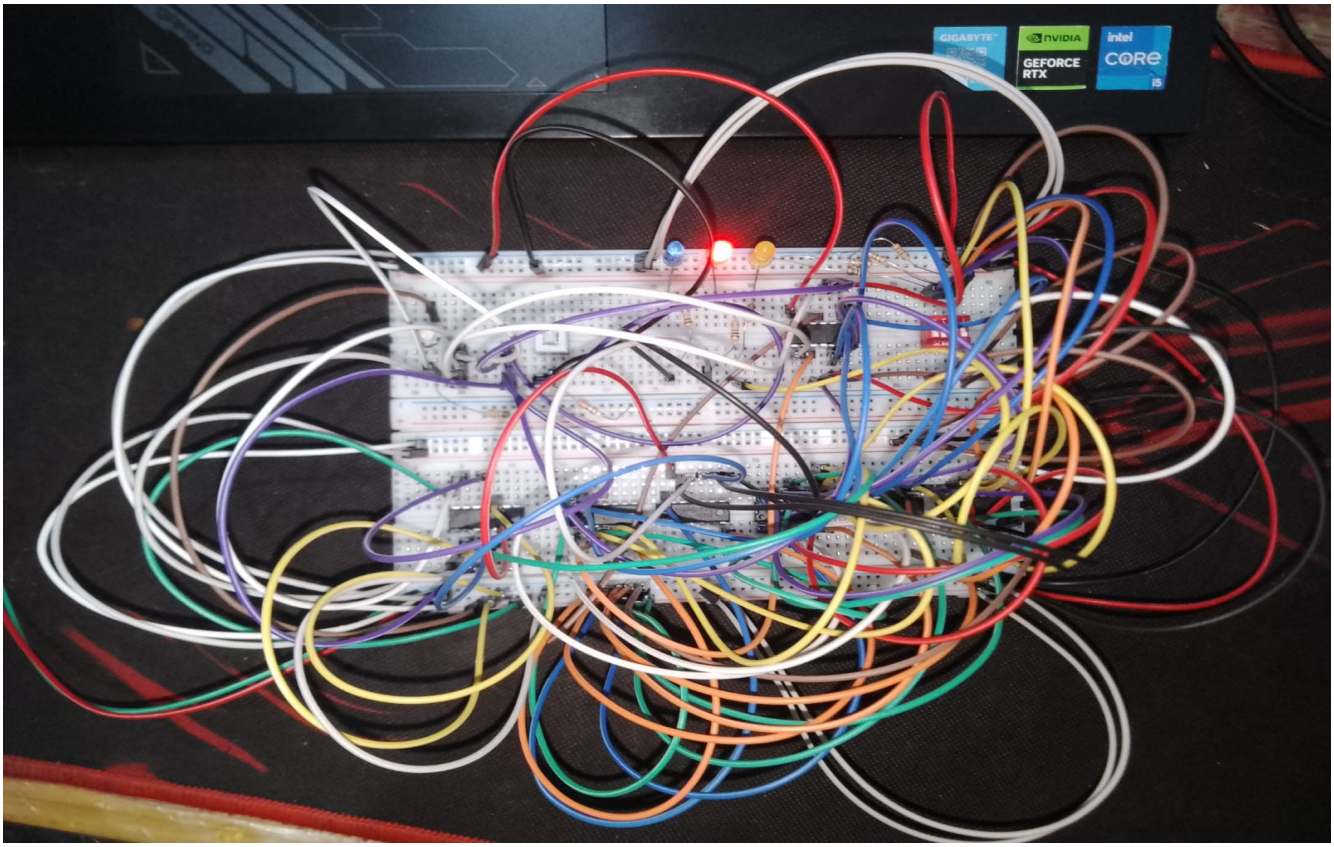


Logisim Implementation:

(a) Logisim



(b) Hardware



The circuit on the breadboard functions as a finite state machine that tracks the number of failed password attempts based on the provided state encoding. Using the Match Logic (Match = S3.S2' .S1'.S0), the system detects the correct 4-bit sequence (1001) and resets the error counter to the initial "0 wrong attempts" state (Q1Q0 = 00). If an incorrect sequence is entered, the Wrong Logic (Wrong = Match') triggers a state transition governed by the D0 and D1 flip-flop input equations. These transitions advance the system through states 01 and 10 until it reaches the final LOCK state (11), as illustrated in Screenshot The physical circuit uses logic gates to compute these boolean expressions and flip-flops to store the current state of the lock.

Problems Faced

- **Set and Reset Logic Wiring:** We encountered significant challenges regarding the **SET and RESET buttons** for the flip-flops. The physical connections for these pins were initially confusing, leading to several instances of wrong wiring that prevented the system from initializing correctly. Debugging these connections was essential to ensure the **RESET signal** could successfully bring the system back to the initial state after a lockout.
- **LED Output Retention:** We faced a problem where the LED outputs (Green and Red) would not stay lit after the ENTER button was released. The system was not properly "remembering" the result of the password check. To fix this, we added **another flip-flop** to the circuit to serve as an output latch. This modification successfully stored the verification result and kept the correct LED active until the next entry.
- **Switch Debouncing:** The ENTER signal, used as the clock for transitions, occasionally caused the state machine to skip counts due to mechanical noise in the push-button. This required careful handling of the button press to ensure the system transitioned correctly through the "wrong attempt" states.

Conclusion

Through this project, I learned the practical application of **sequential logic** and **finite state machines (FSM)**. I gained experience in translating a theoretical state table into a functional hardware circuit using D Flip-Flops and logic gates. This project also highlighted the importance of a RESET mechanism in digital systems to restore the initial state after a security lockout.